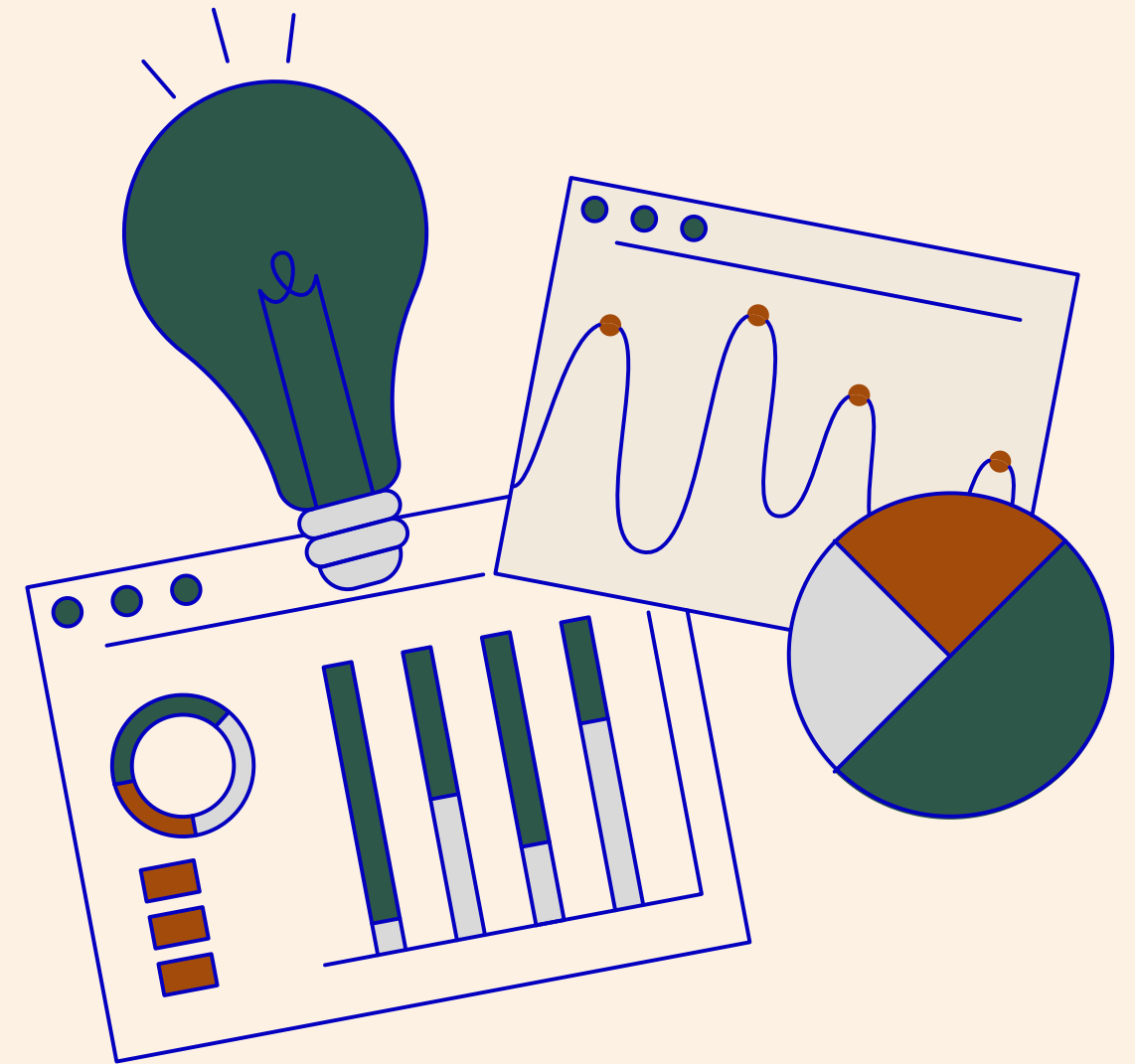
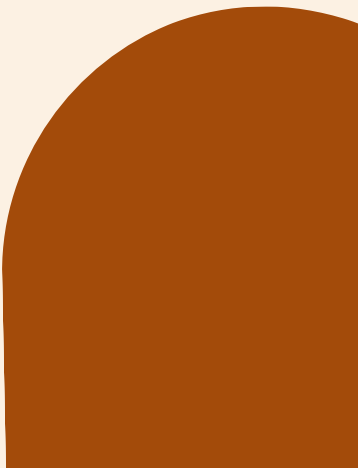
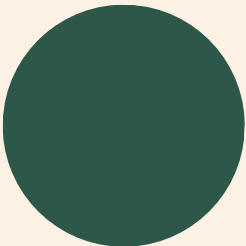
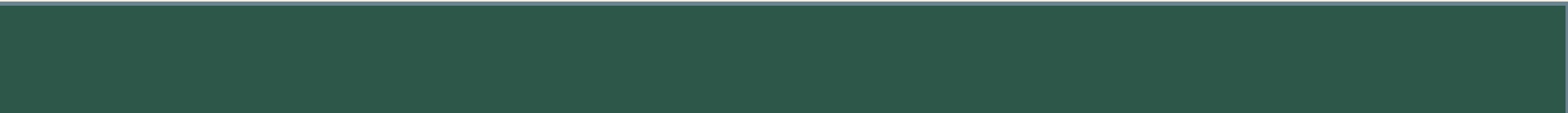
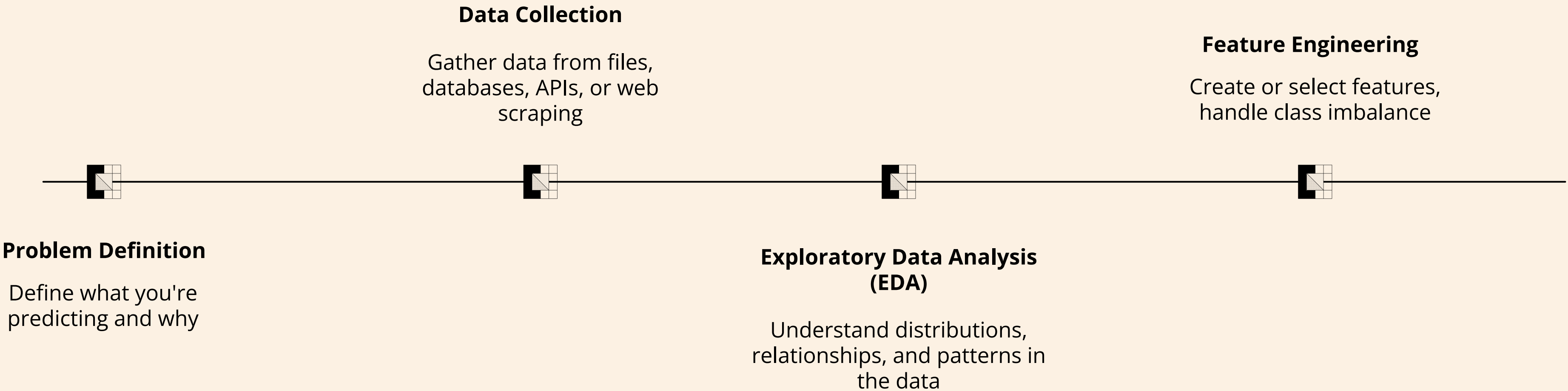
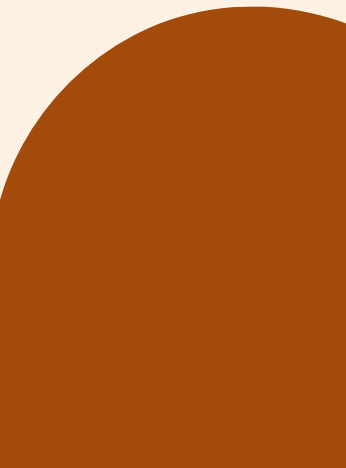
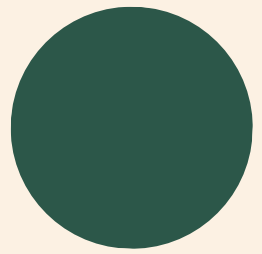
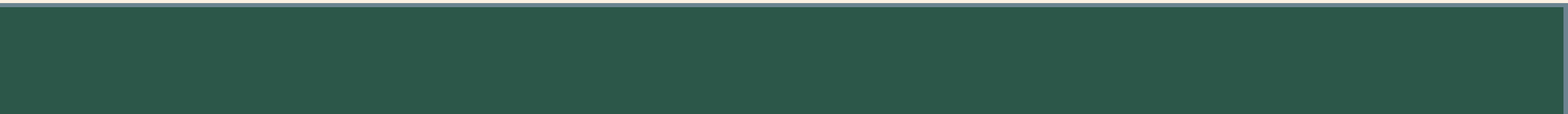
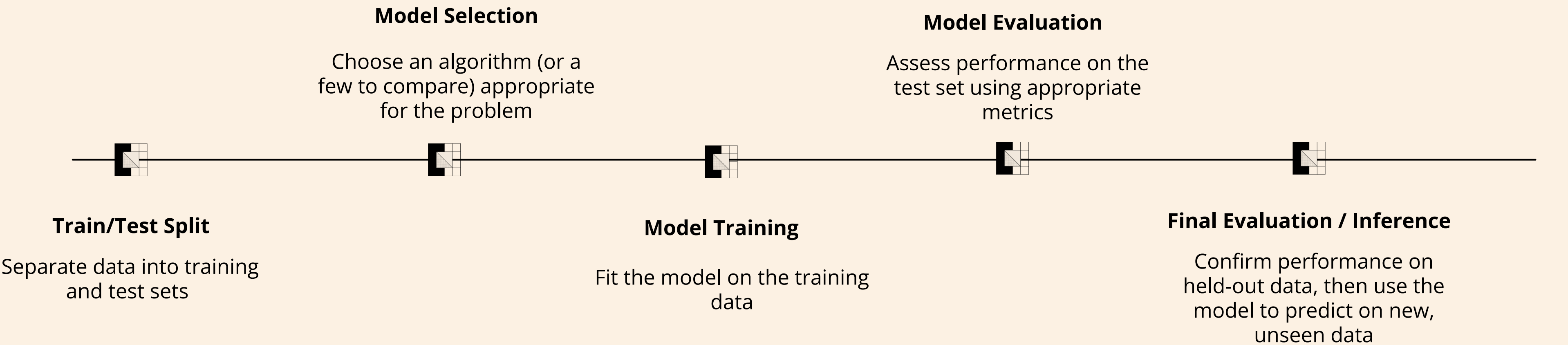


LAB FOR WEEK 7



Machine Learning Process Flow





Feature Scaling

- Feature scaling transforms numerical features so they exist on a comparable range or distribution

	age	income
Person A	25	30000
Person B	45	32000

	age	income
Person A	0.6	0.8
Person B	0.9	0.9

After Scaling

StandardScaler

$$z = \frac{x - \mu}{\sigma}$$

MinMaxScaler

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

from sklearn.preprocessing import StandardScaler

Handling class imbalance

- Class imbalance happens when one or more classes in your target variable have far fewer examples than others
- A model can achieve deceptively high accuracy by simply favoring the majority class and barely learning the minority class at all..

Oversampling (SMOTE)

Generates synthetic examples of the minority class by interpolating between existing minority points

Random undersampling

Removes examples from the majority class to balance things out (risks losing useful data)

```
from imblearn.over_sampling import SMOTE
```

Train/Test Split

- Splitting data into a training set and a test set is how we get an honest estimate of how a model will perform on data it has never seen
- If a model is tested on the exact same data it learned from, it can appear to perform extremely well simply by memorizing patterns specific to that data.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)
```

Model Selection



Model selection is choosing which algorithm to actually train, based on the nature of your problem, your data, and what you need from the result.

Problem type	What you're predicting	Example
Classification	A category / discrete label	Fruit type, spam vs. not spam
Regression	A continuous number	House price, temperature
Clustering	Groupings, with no predefined labels	Customer segments

Common classification algorithms in scikit-learn

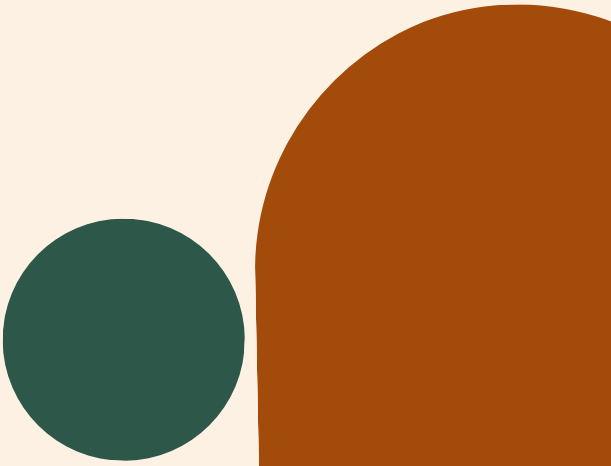
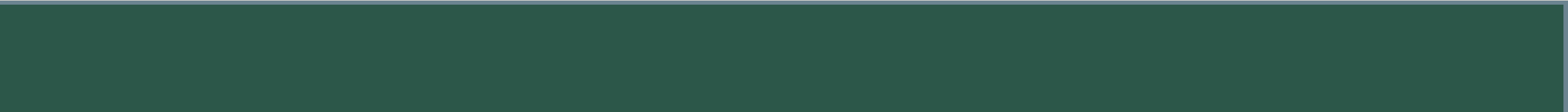
- KNeighborsClassifier
- LogisticRegression
- DecisionTreeClassifier
- RandomForestClassifier
- SVC

Common Regression Algorithms in scikit-learn

- LinearRegression
- Ridge
- Lasso
- DecisionTreeRegressor
- SVR
- KNeighborsRegressor

Common Clustering Algorithms in scikit-learn

- KMeans
- DBSCAN



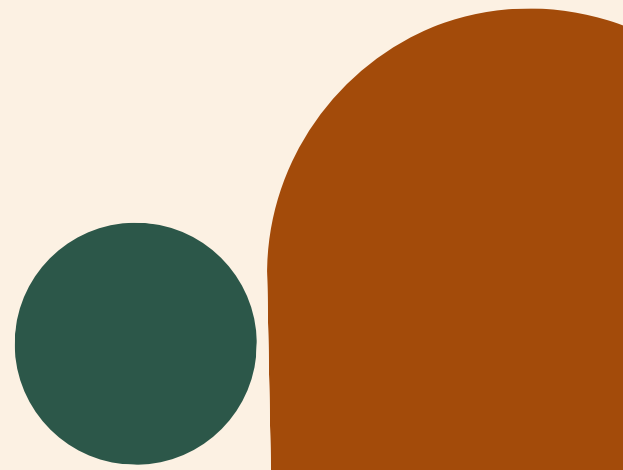
Model Training



Model training is the step where the algorithm actually learns patterns from the training data

```
model = SomeAlgorithm(...) # 1. instantiate, with hyperparameters  
model.fit(X_train, y_train) # 2. train
```

```
knn = KNeighborsClassifier(n_neighbors=5)  
knn.fit(X_train_resampled, y_train_resampled)
```



Model Training



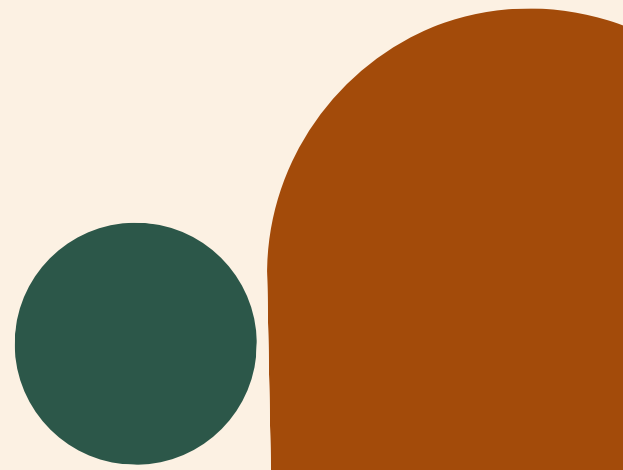
Model training is the step where the algorithm actually learns patterns from the training data

```
model = SomeAlgorithm(...) # 1. instantiate, with hyperparameters
model.fit(X_train, y_train) # 2. train
```

Prediction

Prediction is where a trained model is applied to new data, data it has never seen, to produce an output.

```
predictions = model.predict(X_new)
```



Model Evaluation



- Model evaluation is how we measure whether a trained model actually performs well
- Evaluation differs by problem type

Classification metrics

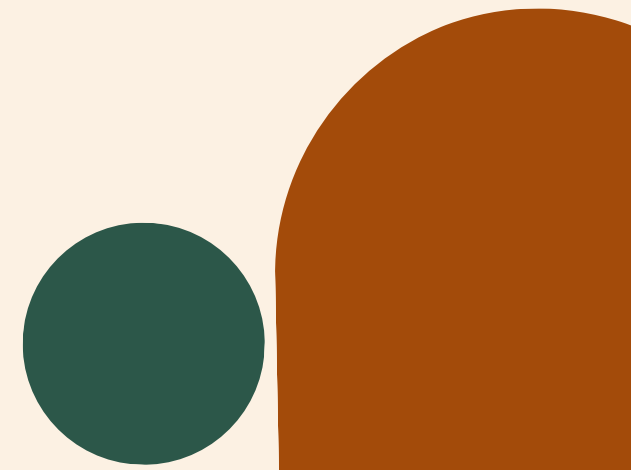
- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score
- Classification Report

Regression metrics

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R^2 Score

Clustering metrics

- Silhouette Score
- Inertia / Elbow Method



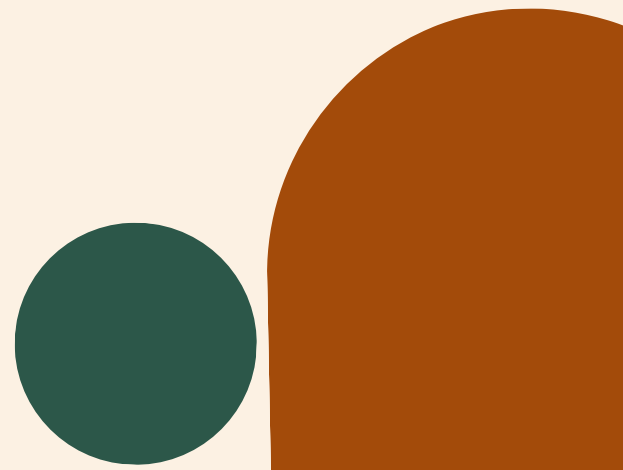
k-Nearest Neighbors (k-NN)



- k-NN is one of the simplest algorithms in machine learning
- It makes predictions by looking at the labeled points closest to a new point, and going with whatever the majority of them say.

The core idea

- Pick a value for k (how many neighbors to consult)
- When a new point arrives, calculate its distance to every point in the training data
- Find the k closest points
- The new point is assigned the class that appears most often among those k neighbors



A new point X arrives at (1.5, 1.5)



Point	Feature 1	Feature 2	Class
A	1	2	🍏 Apple
B	2	1	🍏 Apple
C	1	1	🍏 Apple
D	5	6	🍊 Orange
E	6	5	🍊 Orange
F	5	5	🍊 Orange

Compute distance from X

$\sqrt{(1.5 - 1)^2 + (1.5 - 2)^2} = 0.71$

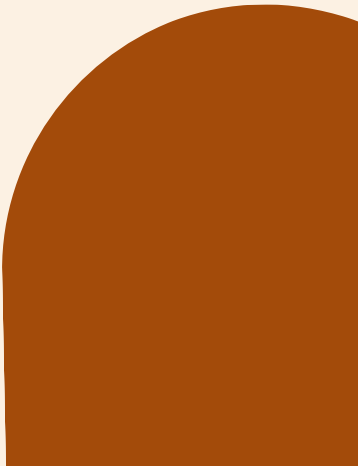
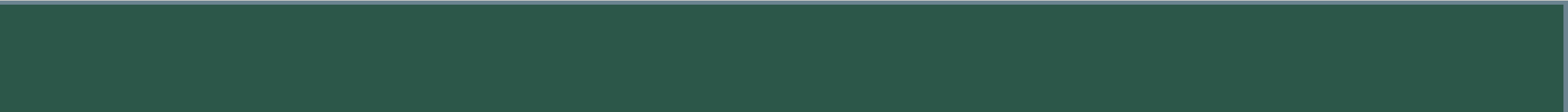


Compute distance from X

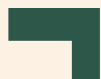
$\sqrt{(1.5 - 2)^2 + (1.5 - 1)^2} = 0.71$



and so on....

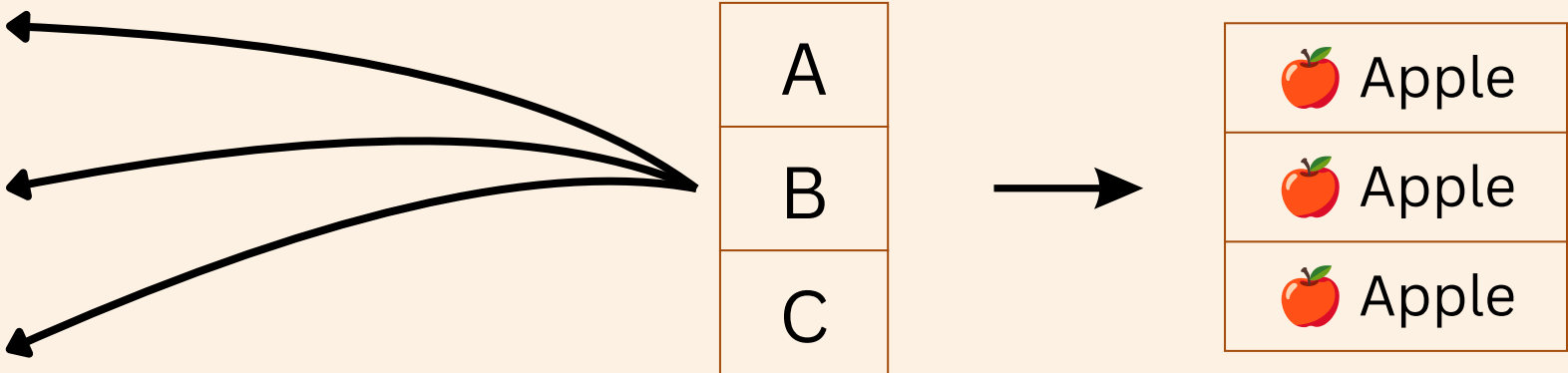


A new point X arrives at (1.5, 1.5)

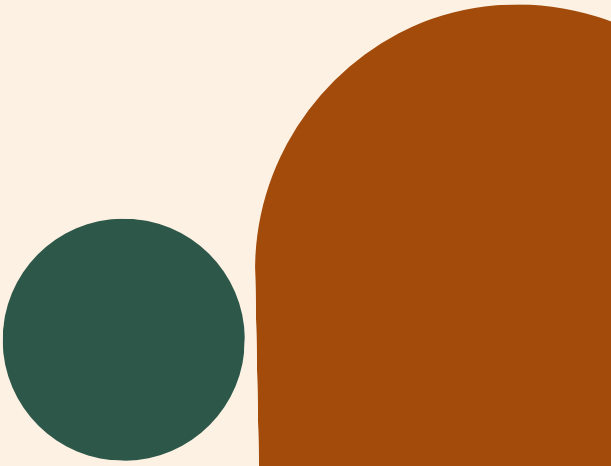
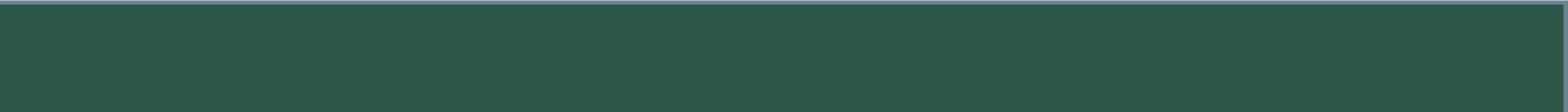


Point	Feature 1	Feature 2	Class	Distance from X
A	1	2	 Apple	0.71
B	2	1	 Apple	0.71
C	1	1	 Apple	0.71
D	5	6	 Orange	5.70
E	6	5	 Orange	5.70
F	5	5	 Orange	4.95

With $k = 3$, we check the smallest three distances



X → 
Apple



LAB FOR WEEK 7

Fruit Classifier using KNN